Compilation Techniques: Laboratory Project
Project Report

Andrea Riolo Vinciguerra

June 2, 2026

Contents

1 Introduction

This project is organized in a single codebase that houses two compilers: MiniImp and MiniFun. Both share common background code (like the diagnostics system) and the core logic is kept strictly separate from the main user interface (via command line). This setup allows users to easily choose between generating a control flow graph, running an interactive interpreter, or executing a code file.

A main focus of the design is modularity. By keeping the code well-organized and adaptable, the system is much easier to maintain and expand over time. Comments are included (more or less helpful) throughout the source files to make comprehension easier.

The code is organized by separating distinct functional blocks into different files. For example, the core evaluator (or interpreter) and the Abstract Syntax Tree definitions for both compilers are stored in a file called `engine.ts`. The rest of the functionality builds upon this file. For example, MiniImp extends this core engine to construct its control flow graph and generate an intermediate representation, while MiniFun supports a complete polymorphic type system.

I chose TypeScript as the programming language for two main reasons. First, I wanted the opportunity to master a language I was less familiar with. Secondly, TypeScript is excellent at handling union types. This feature (used extensively throughout the program) makes writing compiler code much simpler by avoiding the bulky, complex structures often needed in languages like C++ or Rust.

Because Node.js works across different operating systems, the project can be packaged into a single executable for Linux, macOS and Windows. On the latter, however, the user must run the program manually by invoking the `node` command.

2 Running the code

The software can be run in two ways: automatically or manually. The automatic method is designed specifically for Linux or macOS. Instead, the manual approach works on any operating system.

Either approach requires that Node.js and its package manager, `npm`, are installed and properly configured in the system's `PATH`.

2.1 Automatic run

Inside the main project folder, there is a helpful script called `run.sh`. This script handles the setup automatically by downloading the required files, building the chosen compiler (MiniImp or MiniFun), and launching the program.

Running either compiler is straightforward. For example, to start MiniImp, this command is sufficient:

```
$ ./run.sh miniimp --help
```

Similarly, for MiniFun:

```
$ ./run.sh minifun --help
```

Any extra instructions or arguments should be typed directly after the compiler's name. As shown in the examples above, adding the `--help` flag is a great way to see a full list of all the available commands.

2.2 Manual compilation

Manual compilation is available. In such case, these commands must be executed upon downloading the repository:

```
$ npm ci
$ npm run build:miniimp     # or npm run build:minifun
$ dist/miniimp --help       # or dist/minifun --help
```

For Windows, the last step involves invoking `node` explicitly, like so:

```
$ node dist/miniimp --help  # or node dist/minifun --help
```

From now on, the compiler program will be referenced as `./run.sh <compiler>` for consistency.

2.3 Generating MiniImp executables

MiniImp's frontend generates basic LLVM intermediate representation, which cannot be executed directly. To create a runnable program, this IR must be processed by an assembler and a linker.

Before converting the intermediate representation into machine code, it must be optimized. This step removes inefficient patterns, such as the `alloca` trick, which is a technique used for simplifying the generation of LLVM IR. This is explained in detail in a dedicated section.

To make the compilation process easier for Linux and macOS users, a `compile.sh` script is provided. For example, to compile a MiniImp file named `code.mi` into an executable, the following command is used:

```
$ ./compile.sh code.mi
```

The intermediate temporary files are deleted by default, but can be preserved by supplying the argument `-k` before the filename, like so:

```
$ ./compile.sh -k code.mi
```

For manual compilation, standard LLVM tools can be used to generate the final `miniimp` binary:

```
$ ./run.sh miniimp -f code.mi -c >out.ll
$ opt -p=mem2reg out.ll -S -o opt.ll
$ llc -filetype=obj opt.ll -o miniimp.o
$ clang misc/wrapper.c miniimp.o -o miniimp
```

As an additional note, I modified the C wrapper slightly to allow for the arguments to be passed via command line, rather than standard input.

3 Diagnostics system

Both compilers share the same diagnostics system, ensuring consistent and precise error reporting for the user. The interpreters intentionally bypass this system, as runtime evaluation errors represent a failure in code validation rather than a user error.

To provide accurate diagnostics, I made Abstract Syntax Tree nodes track their exact location in the source file using a `SourceSpan`. This structure

records the starting and ending lines and columns without needing to store a copy of the code. When a `DiagnosticError` is thrown, the caller invokes its `format` function and passes in the full source code. Using the `SourceSpan`, this function derives a detailed, colored error message that highlights the exact problem area, along with a clear explanation and any helpful additional notes.

As an example, here is the error presented to the user in case they used an undefined variable in their code:

```
Error: undefined variable 'test'
11 |         c := test + 5;
                 ^^^^
```

The implementation is located in the file `src/diag.ts` and the functionality is used by all the code validators and parsers.

4 MiniFun: the functional language

MiniFun is a functional language supporting three types of first-class values: functions, integers and booleans. It supports let bindings and recursion.

4.1 Abstract Syntax Tree and evaluation

The type definitions for the Abstract Syntax Tree can be found in `src/minifun/engine.ts`. In this file, the type `Value` represents the final result of an evaluation. There are three possible values: `number`, `boolean` and `Closure`. The first two rely on TypeScript's built-in types, while the last one is a custom structure. A basic expression is represented through the type `Expr`, which is a discriminated union of all the available operators.

A closure is a special function that "remembers" its surrounding environment at the time of definition and uses it during execution. Because of this, its type definition must include both the function's body and the saved environment.

As seen in the grammar of the language, some expressions like `let` or `fun` can accept type labels to override automatic type inference. I made these labels optional, giving users the choice to either specify types manually or rely on (and trust) the compiler's type system.

The interpreter processes the Abstract Syntax Tree generated by the parser. It traverses the tree recursively, evaluating sub-nodes, updating the environment and returning the final value. This is what the `evalExpr()` function does.

4.1.1 Environment

Unlike imperative languages, purely functional languages do not allow side effects. Instead of modifying global memory, an evaluated expression returns an updated environment. As the interpreter reaches the end of the tree and climbs back up the recursive chain, these temporary inner environments are discarded until the original (empty) one is reached.

To manage this, I used an `Environment` interface with two main functions: one to look up the value of a specific variable, and another to create a fresh copy of the same environment with a newly assigned identifier. This interface is implemented trivially using a linked list, but is left purposefully generic to allow for more solutions.

4.1.2 Inference rules

The evaluation inference rules are implemented exactly as they are shown in the slides, with the exception of recursive function calls (declared with `letfun`).

Instead of using a special rule to apply recursive functions, which injects the function's name into the environment before evaluating its body, I modified the closure's saved environment to point directly back to the closure object itself. This circular reference allows the recursive function to be called just like any other function, since it is already available within its own stored environment.

The new closure definition rule, therefore, becomes as follows:

$$\text{LetFun} \quad \frac{\rho' = \rho[f \mapsto (f, x, t_1, \rho')] \vdash t_2 \longrightarrow t}{\rho \vdash \texttt{letfun}\ f\ x\ =\ t_1\ \texttt{in}\ t_2 \longrightarrow t}$$

Notice how the function's identifier no longer points to the original environment ρ, but rather to the modified environment ρ'.

4.2 Type system

The type definitions and implementation for MiniFun's type system are located in `src/minifun/validator.ts`. I implemented the complete polymorphic type system in the slides, which is based on the Hindley-Milner algorithm. The whole system revolves around the concept of *monotypes* and *polytypes*, each defined in its own structure.

A monotype either represents a simple concrete type, like integer or boolean, or a generic, undefined type. In such case, the type is represented by a variable. If no extra context is provided, the type stands for itself. An example would be the identity function (`fun x => x`), which has type `a -> a`.

A polytype, on the other hand, is essentially a template waiting to be assigned a more specific type. In the code, this is handled by wrapping a standard monotype inside a structure that keeps track of its bound, yet unassigned variables. This setup allows the system to generate various monotypes from a single polytype, making it possible to reuse the same expression safely in different contexts. As for the example above, the identity function may be defined once and take the types `int -> int` or `bool -> bool` as needed.

4.2.1 Context

Structurally, the type system's context is very similar to the evaluator's environment. However, I modified it to support an iterator operation that can go through all currently defined identifiers. This makes it easier to apply substitutions to the entire context.

Once again, the interface is separated from the actual implementation to favor modularity and code reuse.

4.2.2 Substitutions

A substitution works by mapping a type variable to a specific type or to another variable. In the latter case, it enforces that both generic types must match. For example, a substitution could look like {`a -> int`} or {`b -> a`}.

Different sets can be composed. For instance, to calculate the composition $S2S1$, we must first apply $S2$ to all the monotypes present in $S1$, and then

include any remaining substitution from $S2$. The recursive `composeSubst()` function is designed to do exactly that.

A single substitution pass can update multiple variables at the same time because these mappings are transitive. If multiple variables are linked, they will all resolve to the same final type. For example, given the substitution {`a -> int, b -> a, c -> b`}, the variables `a`, `b` and `c` will all evaluate to `int`.

Because of this transitive behavior, a variable cannot point to itself or a type that contains it. To prevent infinite loops during substitution application, the `bindVar()` function, which handles the assignment of a single type variable, first checks for these circular references.

Substitutions can also be applied to polytypes. Since a polytype is a template that binds certain variables from its inner monotype, the type system only needs to apply substitutions to the remaining free variables. The `freeVarsPoly()` function is responsible for identifying exactly which variables are unbound and available for substitution.

Finally, substitutions can be applied on a whole context. This is done by iterating through the list of all defined identifiers and updating only the relevant (and unbound) type variables.

4.2.3 Algorithm W

The implementation of Algorithm W's inference rules follows the slides. However, I extended these rules to support optional type annotations on `fun` and `letfun` statements. When a type label is omitted, the compiler relies on the standard inference rules. If a label is provided, the system enforces specific type restrictions during the inference process.

Type transformation functions are implemented exactly as per slides:

- `unify()`: solves type constraints by finding a substitution that makes two types identical. It may fail and yield a diagnostic error.
- `gener()`: converts a monotype into a polytype by binding free type variables that are not referenced in the current context.
- `inst()`: replaces bound variables in a polytype with fresh type variables, allowing generic functions to be used with different concrete types

and making polytypes "act" as true templates.

The inference rule for the type-labelled `fun` statement is relatively straightforward. It simply assigns the supplied type to the argument rather than generating a fresh variable:

$$\text{TFun} \quad \frac{\Gamma[x \mapsto \tau] \vdash t : \tau', S}{\Gamma \vdash \texttt{fun}\ x : \tau\ \texttt{=>}\ t : S\tau \to \tau', S}$$

The `letfun` statement requires that the type label be enforced during unification instead, since we supply the whole function type:

$$\text{TLetFun} \quad \frac{\begin{array}{c}\tau' \text{ fresh} \quad \tau'' \text{ fresh} \quad \Gamma[f \mapsto \tau'][x \mapsto \tau''] \vdash t_1 : \tau_1, S_1 \\ S_2 = \text{unify}(S_1\tau', S_1\tau'' \to \tau_1) \quad S_3 = \text{unify}(\alpha \to \beta, S_1\tau'' \to \tau_1) \\ S_3S_2S_1\Gamma[f \mapsto S_3S_2S_1\tau'] \vdash t_2 : \tau_2, S_4\end{array}}{\Gamma \vdash \texttt{letfun}\ f\ x : \alpha \to \beta\ =\ t_1\ \texttt{in}\ t_2 : \tau_2, S_4S_3S_2S_1}$$

5 MiniImp: the imperative language

MiniImp is a simple imperative language that works by receiving an input value through one variable and expecting the program to save its final result into a different variable. The language supports assignments, conditionals, loops and sequences.

5.1 Abstract Syntax Tree and evaluation

The definitions for the Abstract Syntax Tree are located in `src/miniimp/engine.ts`. Unlike MiniFun, the primary building block here is a command (represented by the `Cmd` discriminated union). Because of this imperative design, evaluating a command does not return a final value. Instead, the program operates through side effects, storing its calculations directly in memory. Numeric and boolean expressions are used exclusively within assignments or as comparisons inside conditional statements.

The `evalCmd()` executes commands step-by-step, updating the memory state and evaluating any necessary sub-expressions.

5.1.1 Memory

The `Memory` interface serves as the program's runtime environment. Instead of creating a new copy of the environment for every step, the system uses

a single, shared instance throughout the entire evaluation process. This instance provides two simple methods for managing data: `update()` and `read()`. Trivially, this interface is simply implemented using a standard TypeScript map.

5.2 Control Flow Graph

To analyze the correctness of MiniImp programs, we cannot rely on a standard type-checking system. Because imperative side effects require evaluating all potential branching paths, this problem is instead solved using a *control flow graph*.

All functions related to generating, exporting and manipulating these graphs are located in the file `src/miniimp/graph.ts`.

5.2.1 Minimal graph

A minimal graph is represented by the `Graph` structure, while individual nodes are defined by the `Node` structure. To further restrict operations at the type-system level, derived utility types like `ExitNode` are also included.

There are three types of nodes: `skip`, `assign` and `cond`. Each of these is assigned a specific command from the Abstract Syntax Tree.

The `genGraph()` function inductively generates this minimal graph, ensuring that each block contains at most one instruction. A strict invariant is maintained throughout this process: every sub-graph must have exactly one entry node and one exit node, though these can be the same node.

The algorithm scans the AST and builds a suitable sub-graph for each command while carefully preserving the invariant. Finally, these sub-graphs are linked together using the `seq` pseudo-command.

To respect the invariant at all times, the generator may insert extra `skip` commands not present in the user's original code. These are referred to internally as "fake skips" and are represented as standard `skip` nodes without an associated AST command.

5.2.2 Maximal graph

A maximal graph is represented by the `BlockGraph` structure, with its nodes described by the `Block` structure. Rather than generating a maximal graph from scratch, I created a helper function called `maximizeGraph()` that converts an existing, verified minimal graph into a maximal one.

The structural invariant remains similar, but with an important nuance: the distinction between normal command blocks and "merge" blocks. A merge block functions like a standard block, but specifically incorporates the first "fake skip" following the branches of a conditional statement. I introduced this primarily to keep debugging visualizations clean, as it prevents the graph from generating an empty block when a program ends immediately after an `if` statement. Although merge and standard blocks are functionally equivalent, this specific design safely groups sequential commands while guaranteeing a single exit block for branching statements.

5.2.3 Minimal or maximal?

When determining the ideal graph structure for the compiler, I opted to use the minimal graph as the primary format because it significantly simplifies the analysis and optimization phases. Since each block contains at most one instruction, we can apply straightforward inductive algorithms without needing special heuristics or complex inner-block analyses.

Furthermore, generating a minimal graph requires fewer steps and is more structural, which is exceptionally good for recursive functions.

However, the graph maximizer remains essential for the code generation phase. Grouping instructions into larger blocks allows the compiler to produce clean, readable LLVM IR that is not cluttered with superfluous jump instructions. Additionally, developing this conversion tool was an excellent exercise for my fluency in TypeScript.

5.2.4 Exporting graphs

To assist with debugging and development, I created a generic graph-exporting helper called `graphToDOT()`. This function generates a DOT-formatted string representing the control flow graph. The resulting output can be visualized using online DOT viewers or the standard `dot` command-line utility.

Rather than hardcoding the export logic, this routine accepts a set of callback functions as arguments. Each function plays a specific role in defining the structure and appearance of the exported graph.

These callback functions include:

- `labelShape(node)`: determines the DOT format label and visual shape for a given node.
- `isBranching(node)`: identifies whether the node represents a branching point in the control flow.
- `getNext(node)`: retrieves the subsequent node(s). This can return null at an exit, a single node for sequential commands, or an array of two nodes for conditional statements.
- `skipElem(node)`: evaluates whether a node should be omitted from the visual output, which is particularly useful for hiding internal "fake skip" nodes.

Because of this modular design, `graphToDOT()` can be easily adapted by creating specific wrappers for different graph types. This flexibility allows the compiler to export both minimal and maximal graphs, as well as seamlessly attach extra analysis metadata to each node.

To quickly inspect basic graphs, the module provides two convenient wrappers: `exportGraph()` and `exportBlockGraph()`. A DOT graph can be generated for a specific program by executing the following commands:

```
$ ./run.sh miniimp -f code.mi -g     # minimal graph
$ ./run.sh miniimp -f code.mi -g -m  # maximal graph
```

5.3 Graph analysis

With the generated control flow graph, the compiler must perform data flow analysis to extract information about the execution paths. This process is needed for both verifying code correctness, such as catching undefined variables, and for enabling optimizations.

The logic for these operations is located in the file `src/miniimp/analysis.ts`.

5.3.1 The work queue algorithm

Data flow analysis involves propagating information across the graph until a stable fixpoint is reached. To handle this efficiently, I implemented a generic `wqAnalyze()` function.

This method uses a queue-based breadth-first search algorithm. It enqueues graph nodes and applies a custom local update function to each one. If a node's internal state changes during this update, its neighboring nodes (either successors or predecessors, depending on the direction of the analysis) are added back to the work queue.

The `wqAnalyze()` generic wrapper reduced code duplication when implementing multiple distinct analyses, and is driven by two primary callback functions provided as arguments:

- `init(node)`: establishes the initial state for a single node before the main analysis begins. This function is executed once for every node in the graph, relying on the surrounding scope to set up the necessary data structures.
- `local(node, ctx, wq, map)`: defines the core update logic applied to each node as it is processed from the work queue. It receives the current node, a context object (`ctx`) containing its immediate neighbors, the current state of the analysis (`map`), and a reference to the work queue (`wq`). Providing direct acces to the queue allows the function to re-enqueue neighboring nodes, ensuring that state changes are propagated across the graph until a stable state is reached.

5.3.2 Code specifics and types

All the analyses work by assigning a special structure to each graph node via a standard TypeScript map. For the *Defined* and *Live Variables* analyses, the structure `VarSets` is used (aliased to `DefinedVars` and `LiveVars`, respectively), which is a pair of identifier sets: one for the *in* variables and one for the *out* variables.

The *Reaching Definitions* analysis, on the other hand, uses the `DefSets` structure, which stores the nodes where the assignment occurs by reference. This avoids having to keep a separate map in memory to assign definition nodes a unique identification number.

5.3.3 Defined Variables and validation

The function `analyzeDefinedVars()` handles the most critical analysis, the *Defined Variables*. Using the output from this process, `checkUndefined()` scrubs the graph to ensure that no command ever attempts to read a variable before it has been assigned a value. If an uninitialized variable is detected, the validation process fails and the compiler throws a `DiagnosticError` with the precise location of the error in the source code.

5.3.4 Live Variables and Reaching Definitions

The *Live Variables* and *Reaching Definitions* analyses are handled by two other functions: `analyzeLiveVars()` and `analyzeReaching()`. These match the specifications in the slide exactly.

5.3.5 Graph visualization

To aid in debugging and verify the correctness of the analyses' logic, the module includes two wrappers of the generic `graphToDOT()` graph exporting helper.

`exportDefinedVars()`, `exportLiveVars()` and `exportReachingDefs()` convert the internal graph structures into DOT format strings that display the calculated *in* and *out* data sets for every single node in the control flow graph.

To generate a DOT graph for a specific analysis, these commands may be executed:

```
$ ./run.sh miniimp -f code.mi -g -ad  # defined variables
$ ./run.sh miniimp -f code.mi -g -al  # live variables
$ ./run.sh miniimp -f code.mi -g -ar  # reaching definitions
```

5.4 Optimization

Once analyses have been performed on the control flow graph, the compiler enters the optimization phase. The goal of this step is to produce a more efficient graph by stripping away redundant instructions and pre-calculating predictable values at compile time, before handing off the final result to the code generation phase.

The optimization logic is housed in `src/miniimp/opt.ts` and consists of three distinct passes that work together to simplify the graph.

5.4.1 Dead Store Elimination

Dead Store Elimination is a structural optimization that removes useless assignments from the code. The pass is implemented in the `dse()` function according to the slides, and relies on the *Live Variables* analysis.

The algorithm iterates through all the nodes in the control flow graph. If it encounters an `assign` node, it checks the `liveOut` set for that block. If the assigned variable does not appear in the *out* set, it means that the variable is never read by any subsequent instruction. Therefore, the assignment is completely redundant and can be safely deleted.

Once again, the advantage of using a minimal graph is evident: because a graph node can only contain one instruction, there's at most one assigned variable for every single node.

Because removing a node structurally alters the graph, the predecessor and successor links must be re-routed to avoid breaking the graph's invariants. After a successful deletion, the graph is permanently changed, meaning the data flow maps are now stale. Therefore, as seen in the code's comments, the DSE pass requires that all analyses be re-run, and the pass repeats until no further dead stores are found.

5.4.2 Constant Folding

Constant Folding is a local optimization that simplifies expressions before runtime. The `constFold()` function traverses both numeric and boolean expressions within the Abstract Syntax Tree.

When the compiler detects an operation involving only raw literal values, it evaluates the result immediately and replaces the entire sub-tree with a single value node.

The numeric folding logic incorporates arithmetic identities, such as stripping away `x + 0` or `x * 1`, and converting `x * 0` to `0`. It even performs simple associative folding, automatically reducing expressions like `10 + (20 + x)` directly down to `30 + x`.

The functions responsible for the folding are `foldNumExpr()` and `foldBoolExpr()`. These return a `MergeResult` object, which contains the final expression and whether it is different from the original. This information is needed because the AST node may or may not be the same.

I thought of creating a more complex algebraic system to keep track of the variable factors used in mathematical formulas, so as to simplify expressions such as `x + 3 * y + 5 * x + 2 * y` into `6 * x + 5 * y`, but that seemed slightly off the scope of the project.

Unlike DSE, Constant Folding only mutates the inner AST nodes and never alters the structural layout of the graph itself.

5.4.3 Constant Propagation

Constant Propagation complements Constant Folding by replacing variables with their known literal values. It is implemented via `constProp()` and follows the slides exactly, leveraging the *Reaching Definitions* analysis.

Whenever an expression uses a variable, the compiler checks the *in* set of reaching definitions for that specific node. If there is exactly one assignment reaching the node, and that assignment sets the variable to a raw literal value, the compiler replaces the variable reference with the literal itself.

Similarly to DSE, executing this replacement alters variable usage throughout the graph, requiring data flow maps to be recalculated before the next pass.

5.4.4 The optimization loop

These three passes work side by side to provide an ultimately optimized graph. For example, applying Constant Propagation might expose new literal values that can be squashed together via Constant Folding. This folded constant might then cause a previously useful variable to become obsolete, triggering Dead Store Elimination.

To maximize efficiency, the main `optimize()` function orchestrates these passes using a fixpoint iteration loop. It continuously cycles through `doDSE()`, `doConstFold()` and `doConstProp()` in sequence. The loop only terminates when a full cycle completes without any of the three passes reporting a change to the graph or the AST, guaranteeing that the code is as optimized as possible before code generation.

5.4.5 Graph visualization

The compiler supports generating a DOT graph after optimizations have been applied. To do so, the following command is available:

```
$ ./run.sh miniimp -f code.mi -g -o
```

It is possible to use the `-o` flag together with analysis-specific graph generation flags, like `-ad`.

5.5 Code generation

The final phase of the compiler is to translate the verified and optimized control flow graph into an intermediate representation. The choice of the project is to rely on LLVM for it.

The entry point for this is the `codegen()` function. Before emitting any code, the compiler guarantees correctness by running the validation and optimization passes, and then uses the `maximizeGraph()` helper to group the minimal nodes into larger basic blocks, which is ideal for LLVM IR generation as seen previously.

5.5.1 Variable allocation and the `alloca` trick

A difficult problem when generating LLVM IR directly is adhering to its strict *Static Single Assignment* form, which requires the complex calculation of *phi* nodes for variables that change values across different control flow branches. While the maximal graph may come in handy for this specific type of problem, I ultimately decided to bypass this and implement what is commonly known as the `alloca` trick, as advised in the slides. This is used for example by the Rust compiler as well.

By invoking the `extractAllVars()` helper, the compiler identifies every unique identifier used throughout the program, including input, outputs and intermediate variables. At the very beginning of the LLVM function definition, a dedicated memory slot of the stack is allocated for each of these variables using the `alloca` instruction.

This architecture allows the rest of the generated code to safely update variables using standard `load` and `store` operations, deferring the SSA conversion to LLVM's external `mem2reg` optimization pass.

5.5.2 Expression translation

To translate individual operations into machine-level instructions, the compiler relies on two recursive helper functions: `genNumExpr()` for aritmetic and `genBoolExpr()` for boolean logic.

These functions behave very similarly to the interpreter found in the `engine.ts` module, traversing the expression trees bottom-up. For every subexpression evaluated, the compiler allocates a new, sequentially numbered temporary LLVM register (e.g., `%1`, `%2`). Variables are loaded from their stack pointers into these temporary registers, and mathematical operations are mapped directly to their LLVM equivalent (such as `add`, `sub`, or `mul`).

For boolean logic, the compiler utilizes `and`, translates the `not` operator into a `xor` instruction against the value 1, and evaluates less-than comparisons using the `icmp slt` opcode.

5.5.3 Graph traversal and block emission

The final structure of the LLVM IR is constructed using a depth-first search algorithm over the maximal graph, implemented via the `traverse()` inner function. As the routine visits each block, it assigns a unique label (such as `block_0`) to define a proper LLVM basic block.

Within each block, sequential assignment commands are evaluated, and their resulting temporary registers are stored directly into stack memory. For conditional blocks, the resulting condition value is stored into a register, which is then used in the conditional `br` instruction.

In LLVM, every basic block must strictly terminate with a branching or returning instruction. The `traverse()` crawler handles this dynamically based on the block's type:

- **Conditional blocks** evaluate their boolean condition and emit a conditional branch instruction (`br i1`) that jumps to either the true or false block labels.
- **Standard blocks** that have a subsequent step emit an unconditional branch (`br`) to their single successor node.
- **Exit blocks** load the final computed value from the program's designated output variable and emit a `ret` instruction to return that integer

to the caller.